

RIFAR: Reliability and Forgetting-Aware Replay for Continual Robot Learning

Zirong Song¹, Zheng Lu², Haoran Liao², Wanqi Zhong²

Yunhe Ni², Lijie Wang^{2,4}, Xiuying Chen^{1,†}

¹ MBZUAI ² Tsinghua University ⁴ Zhejiang University

Abstract

Genuine embodied agency requires robots to turn continuous real-world experience into lasting, transferable skills. This demands continual learning that integrates new capabilities without eroding prior knowledge as tasks and environments evolve. Experience replay mitigates forgetting, but storing complete demonstrations becomes costly as tasks accumulate. World-action models offer a generative alternative, reconstructing past experience through joint predictions of actions and future observations. However, visually coherent rollouts may contain actions that cannot realize the predicted transitions, while new-task adaptation can disrupt previously learned behavior. RIFAR therefore combines reliability screening with drift-aware replay selection. It reconstructs trajectories from compact demonstration prefixes and uses a frozen inverse-dynamics model to assess action-visual consistency. Training first combines current demonstrations with the highest-quality screened trajectories. RIFAR then compares action predictions before and after this adaptation on identical historical inputs, re-selecting trajectories with larger normalized drift from the same screened pool for continued training. Across three LIBERO suites and real-world experiments, RIFAR surpasses the previous state of the art in WAM-based generative replay. On LIBERO-Goal, it achieves 90.97 AUC while retaining only 320 historical time steps per task, approximately 4.9% of the steps retained using 50-demonstration replay.

1 Introduction

Robots operating in evolving environments must continually acquire new manipulation skills while retaining earlier capabilities. Yet adapting a shared action policy to new tasks can disrupt previously learned behavior, causing catastrophic forgetting (Kirkpatrick et al., 2017; Liu et al., 2023). Experience replay mitigates this problem by training on stored demonstrations alongside current-task data

(Chen et al., 2026; Zhao et al., 2026). However, these demonstrations contain long sequences of observations, proprioceptive states, and actions, making storage increasingly costly as tasks accumulate. Generative replay reduces this storage burden by reconstructing historical training examples from model knowledge (Shin et al., 2017). World-action models (WAMs) are well suited to this approach: by jointly predicting actions and future observations (Zhu et al., 2025), they can recurrently reconstruct past-task trajectories for replay. Despite this potential, continual learning with WAMs remains largely unexplored. REGEN takes an initial step by generating replay trajectories from historical task instructions and current-task observations for continual imitation learning (Govind et al., 2026).

Although REGEN demonstrates the feasibility of WAM-generated replay, its storage-efficient design does not ensure reliable supervision. To avoid retaining historical visual contexts, REGEN conditions generation on a past-task instruction and a current-task observation. However, the current scene may lack the objects or spatial configuration required for the earlier task. Retaining compact historical context provides a task-compatible starting point, but does not eliminate errors during recurrent generation. A WAM may still produce a visually coherent rollout whose actions fail to realize the depicted transitions, resulting in misleading replay supervision. Reliability screening, however, does not determine which remaining trajectories are most important for preserving earlier skills. Learning a new task can induce substantial action drift on some historical trajectories, potentially disrupting previously successful behavior. These trajectories warrant greater replay priority to reinforce affected behaviors during further training. RIFAR therefore first screens generated trajectories for action-visual consistency, then prioritizes reliable trajectories with larger action drift to preserve the earlier behaviors most affected by new-task learning.

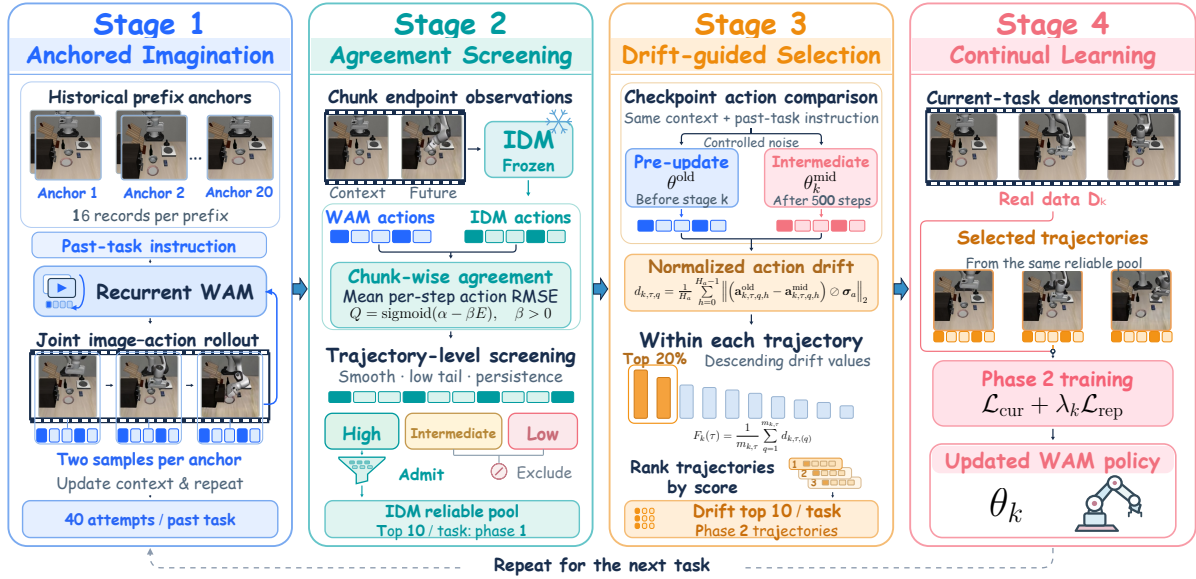


Figure 1: RIFAR overview. For each historical task, twenty 16-step prefixes and two seeds per prefix provide forty generation attempts. After failed generations are removed, a frozen IDM screens the remaining candidates. Phase 1 combines current-task demonstrations with the ten highest-quality admitted trajectories for 500 steps. Action drift between the pre-update and intermediate policies then guides reselection of ten trajectories from the same reliable pool for continued training. The largest-20% mean is computed within each candidate trajectory.

We introduce **RIFAR (Reliability and Forgetting-Aware Replay)** for continual WAM learning (Figure 1). RIFAR retains only 320 historical time steps per task—4.9% of the data stored by 50-demonstration experience replay—to initialize recurrent generation of past-task trajectories. A frozen inverse-dynamics model screens these trajectories for consistency between their actions and predicted visual transitions. In the first phase, the policy trains on current-task demonstrations and the highest-quality screened trajectories. RIFAR then compares action predictions on identical historical states before and after this phase to identify earlier behaviors most affected by adaptation. It reselects trajectories from the same screened pool for the second phase, prioritizing those with the largest action shifts. This targeted replay reinforces earlier behaviors while the policy learns the new task. We evaluate retention through success rates on previously learned tasks.

We evaluate RIFAR using the Cosmos Policy backbone on LIBERO-Goal, LIBERO-Object and LIBERO-Spatial. RIFAR surpasses REGEN by 23.17, 5.39 and 9.93 AUC points on the three suites, respectively. We further examine the trade-off between historical storage and performance on LIBERO-Goal. RIFAR reaches 90.97 AUC while retaining only 320 historical time steps per task, outperforming ten-demonstration experience re-

play (ER(10)), which achieves 90.70 AUC with approximately 1,300 retained steps. We additionally validate RIFAR on a real Franka robot across three manipulation tasks, achieving 57.78 AUC and demonstrating the effectiveness of our approach in a physical-world continual learning scenario with diverse manipulation behaviors and sequential task transitions. Ablation studies and action-drift analyses further isolate the contributions of reliability screening and adaptive replay selection to preserving previously learned manipulation skills.

Our main contributions are as follows:

- **Reliable generative replay for WAMs.** RIFAR reconstructs past-task trajectories from compact context and uses a frozen inverse-dynamics model to reject trajectories whose actions are inconsistent with depicted visual transitions.
- **Forgetting-aware replay selection.** RIFAR identifies the earlier behaviors most affected by learning a new task by comparing action predictions before and after initial adaptation. It then prioritizes reliable trajectories with the largest action changes for a second phase of training.
- **Valid Experiments.** Experiments on LIBERO-Goal and LIBERO-Object demonstrate substantial improvements over REGEN, while ablations isolate the contribution of each component and storage analysis shows strong performance with only 320 historical time steps retained per task.

2 Related Work

2.1 Continual Learning

Continual learning aims to acquire new capabilities while retaining prior knowledge. EWC constrains updates according to Fisher-based parameter importance (Kirkpatrick et al., 2017), while PackNet preserves task-specific weights through iterative pruning and masking (Mallya and Lazebnik, 2017). LoRA reduces trainable parameters through low-rank updates to frozen pretrained weights (Hu et al., 2021), but parameter efficiency alone does not guarantee retention. Experience replay revisits earlier tasks by mixing past examples with current-task data (Liu et al., 2023). Complete manipulation demonstrations contain sequences of images and actions. Generative replay instead reconstructs historical training examples (Shin et al., 2017), reducing reliance on stored demonstrations.

2.2 Vision-Language-Action Models for Robot Manipulation

Recent work applies continual learning to vision-language-action (VLA) policies (Lu et al., 2026). CLARE expands adapters using feature novelty and routes inputs without task labels or stored exemplars (Römer et al., 2026). For continual reinforcement learning, CRL-VLA combines goal-conditioned value modeling with asymmetric regularization (Zeng et al., 2026), while Simple Recipe Works finds limited forgetting when adapting pretrained VLAs with LoRA and on-policy reinforcement learning (Hu et al., 2026). Replay mitigates forgetting in real-world VLA experiments (Zhu et al., 2026). PHASER allocates replay capacity by manipulation phase and prioritizes samples through language-vision similarity and action divergence (Chen et al., 2026); Info-VLA uses replay-anchored contrastive learning with cross-modal mutual-information constraints (Zhao et al., 2026).

2.3 World Action Models for Continual Robot Learning

World-action models jointly predict actions and future observations, enabling them to recurrently generate complete manipulation trajectories (Lu et al., 2026; Kim et al., 2026). This capability provides a natural basis for generative replay, as earlier experience can be reconstructed by the model rather than stored in full. Despite this potential, continual learning with WAMs remains largely unexplored. REGEN represents an initial approach,

training on generated trajectories from earlier tasks alongside current-task data (Govind et al., 2026). To reduce historical storage, its main configuration conditions earlier-task instructions on current-task observations. This may start generation from a scene that lacks the objects or spatial configuration required by the earlier task. Moreover, a coherent predicted video does not guarantee that its paired actions would produce the depicted transitions during robot execution. Although REGEN documents these inconsistencies, it does not explicitly remove unreliable trajectories before replay or prioritize trajectories according to how strongly new-task learning changes earlier action predictions.

3 Method

RIFAR combines reliability screening with adaptive replay selection for continual robot learning with WAMs. As shown in Figure 1, it reconstructs past-task trajectories from compact historical contexts and screens them for action-visual consistency to form a reliable replay pool. Training proceeds in two phases. We first adapt the policy using current-task data and the highest-quality screened trajectories. We then compare its action predictions before and after this adaptation on identical historical contexts from the same replay pool. Trajectories with the largest normalized action drift are prioritized for replay in the second phase, while training on the current task continues.

3.1 Problem Setup

We study continual robot learning, where a single policy learns N manipulation tasks, denoted by $T_0, \dots, T_{N-1}$, over successive stages. For each task T_i , a language instruction ℓ_i describes the goal, and a demonstration dataset $\mathcal{D}_i$ provides example trajectories for completing the task. Each demonstration trajectory records a sequence of visual observations, proprioceptive states, and robot actions.

After initialization, each incremental stage introduces a single new manipulation task. We denote the introduction stage of task T_i by $g(i)$ and the demonstration dataset of the new task at stage k by $\mathcal{D}_k^{\text{new}}$. For historical tasks with $g(i) < k$, complete demonstrations are no longer available; we retain only compact historical contexts from earlier demonstrations for replay generation. Before adaptation, we save the previous policy checkpoint as $\theta_k^{\text{old}} = \theta_{k-1}$ to measure subsequent action drift on identical past-task contexts. The policy is then

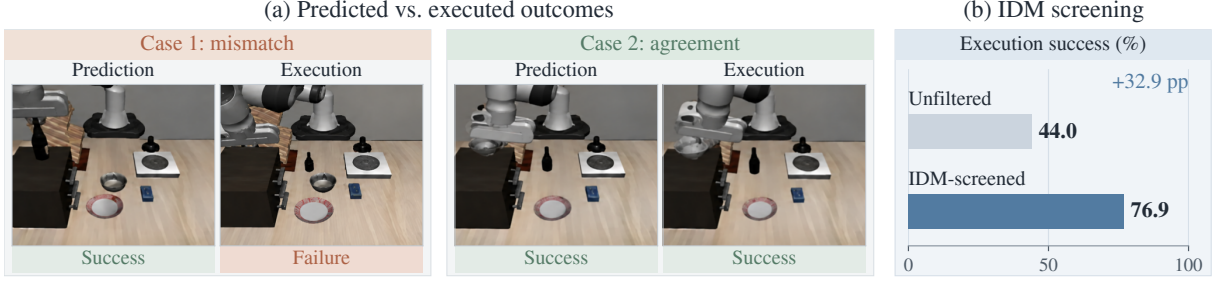


Figure 2: IDM reliability screening. (a) Predicted outcomes and simulator executions: one case fails despite predicted success, whereas the other is consistent. (b) Execution success is 44.0% among unfiltered trajectories and 76.9% among IDM-screened trajectories, a difference of 32.9 percentage points.

trained on new-task demonstrations together with generated historical replay that passes screening. After each stage, we evaluate the updated policy on the new task and all previously learned tasks to assess both task acquisition and skill retention.

3.2 Initial-State Anchored Historical Imagination

Previous WAM-based generative replay reconstructs past-task trajectories by jointly predicting actions and future observations. Without historical context, however, generation may begin from a scene that lacks the objects or spatial configuration required by the task. We therefore introduce **Initial-State Anchored Historical Imagination**, which retains short sequences of observations and proprioceptive states from the beginning of selected historical demonstrations. These sequences provide task-compatible starting points for generation without requiring complete demonstrations to be stored. For each historical task T_i , we select the first K demonstrations in dataset order and retain only the initial H_s time steps of each selected demonstration. These short sequences form the anchor set $S_i = \{s_{i,j}\}_{j=1}^K$. Each anchor is defined as $s_{i,j} = \{(o_t, p_t)\}_{t=0}^{H_s-1}$, where o_t and p_t denote the visual observation and proprioceptive state at time step t in the corresponding demonstration. We use $K = 20$ and $H_s = 16$, storing only $KH_s = 320$ observation–proprioception pairs per task. During generation, the WAM takes the current observation o_t , proprioceptive state p_t , and task instruction ℓ_i as input and jointly predicts an H -step action chunk and the corresponding future state at $t + H$:

$$(\hat{A}_t, \hat{o}_{t+H}, \hat{p}_{t+H}) = G_\theta(o_t, p_t, \ell_i; \xi), \quad (1)$$

where $\hat{A}_t = (\hat{a}_t, \dots, \hat{a}_{t+H-1}) \in \mathbb{R}^{H \times d}$ contains H consecutive action predictions, each of dimension d . The outputs $\hat{o}_{t+H}$ and $\hat{p}_{t+H}$ are the pre-

dicted visual observation and proprioceptive state at time $t + H$. Here, θ denotes the WAM parameters, and ξ represents sampling randomness.

Generation begins with a retained anchor, whose observation–proprioception pairs are supplied to the WAM in temporal order. At each generation step, the model predicts an action chunk and a corresponding future state, and the first action $\hat{a}_t$ is appended to the replay trajectory. Once the stored sequence is exhausted, the WAM continues generation using observations and proprioceptive states predicted in earlier steps. This recurrent process extends the short historical context into a long-horizon candidate replay trajectory.

We denote the b -th generated rollout from the retained anchor $s_{i,j}$ by $\hat{\tau}_{i,j,b} = \{(\hat{o}_t, \hat{p}_t, \hat{a}_t)\}_{t=0}^L$, where L is the final time index. All rollouts generated from the anchors of historical task T_i form the candidate replay pool $\mathcal{P}_i^{\text{cand}} = \{\hat{\tau}_{i,j,b}\}_{j,b}$ for subsequent IDM-based reliability screening.

3.3 Reliability Screening and Initial Replay

Even with historical anchors, WAM-generated actions may be inconsistent with the predicted visual transitions. To assess this consistency, we use an inverse-dynamics model (IDM) I_ϕ that combines a frozen DINOv2 ViT-S/14 visual encoder with a two-layer Transformer decoder. Given the paired initial and endpoint images, the IDM predicts a 16-step chunk of seven-dimensional actions. We train it on the original demonstrations from LIBERO Tasks 0–5, using image pairs 16 time steps apart as visual inputs and the intervening action sequence as supervision. After training, the entire IDM remains frozen throughout continual learning.

For each generated chunk r , let $\hat{o}_r$ and $\hat{o}_r^+$ denote its initial observation and predicted endpoint observation, respectively. The IDM infers the action chunk as $\hat{A}_r^{\text{inv}} = I_\phi(\hat{o}_r, \hat{o}_r^+)$. We normalize

both the IDM-inferred and WAM-generated actions using the same transformation $\tilde{a} = \mathcal{N}(a)$ and compute their chunk-level disagreement as:

$$E_r^{\text{IDM}} = H_a^{-1} d^{-1/2} \sum_{h=0}^{H_a-1} \|\tilde{a}_{r,h}^{\text{inv}} - \tilde{a}_{r,h}\|_2. \quad (2)$$

Here, H_a is the number of actions in each compared chunk, and d is the action dimension. At position h in chunk r , $\tilde{a}_{r,h}^{\text{inv}}$ and $\tilde{a}_{r,h}$ denote the normalized IDM-inferred and WAM-generated actions, respectively. Thus, E_r^{IDM} measures the root-mean-square error across action dimensions at each step, averaged over the chunk. We convert this disagreement into an agreement score:

$$Q_r^{\text{IDM}} = \sigma(\alpha - \beta E_r^{\text{IDM}}), \quad \beta > 0, \quad (3)$$

where $\sigma(z) = (1 + \exp(-z))^{-1}$ is the sigmoid function, and α and β control the offset and sensitivity of the mapping. The constraint $\beta > 0$ ensures lower disagreement yields higher scores.

We smooth chunk scores with a three-chunk trailing average and define $Q(\hat{\tau})$ as the mean of the lowest 20% of smoothed scores. Screening also rejects trajectories with persistent low agreement (Appendix C.2). WAM-generated actions remain the targets for replay. For each historical task T_i , trajectories that pass screening form the replay pool $\mathcal{P}_{k,i}$. We select the ten highest-scoring trajectories from each pool for Phase 1 and denote the selected set by $\mathcal{S}_{k,i}^{(1)}$. Starting from θ_k^{old} , we train the policy for 500 optimization steps using current-task demonstrations and the selected replay trajectories from all historical tasks, yielding the intermediate checkpoint θ_k^{mid} . The full screened replay pools remain available for reselection in Phase 2.

3.4 Drift-Based Replay Prioritization

To guide replay reselection in Phase 2, we compare the action predictions of the pre-update policy θ_k^{old} and the intermediate policy θ_k^{mid} , obtained after 500 optimization steps, on contexts from the full screened replay pools. Both policies receive identical observations, proprioceptive states, and task instructions under the same inference settings. For each evaluated context q in trajectory $\hat{\tau}$, both policies predict an H_a -step action chunk. With stage k , trajectory $\hat{\tau}$, and context q fixed, we denote their actions at chunk position h by a_h^{old} and a_h^{mid} . We average the normalized action distances over the chunk to obtain the context-level drift:

$$d_{k,\tau,q} = H_a^{-1} \sum_{h=0}^{H_a-1} \|(a_h^{\text{old}} - a_h^{\text{mid}}) \oslash \sigma_a\|_2, \quad (4)$$

where $\oslash$ denotes element-wise division and the fixed positive vector $\sigma_a \in \mathbb{R}_+^d$ provides a shared normalization scale for each action dimension.

For each trajectory $\hat{\tau}$ with M_τ evaluated contexts, let $d_{k,\tau,(q)}$ denote its q -th largest context-level drift. We use $m_\tau = \text{Round}_{\text{tail}}(0.2M_\tau)$, with $1 \leq m_\tau \leq M_\tau$, to denote the nonempty number of contexts in the upper 20% tail. The forgetting-aware replay priority is their mean drift:

$$F_k(\hat{\tau}) = m_\tau^{-1} \sum_{q=1}^{m_\tau} d_{k,\tau,(q)}. \quad (5)$$

The upper-tail mean emphasizes localized policy changes that can be diluted by averaging over the entire trajectory. For each historical task T_i , we select the ten highest-priority trajectories from the same full screened pool $\mathcal{P}_{k,i}$. This set replaces the Phase 1 replay set, with possible overlap. Starting from θ_k^{mid} , we continue training for 300 optimization steps using current-task demonstrations and the reselected replay trajectories, yielding the updated policy θ_k . The two phases together comprise 800 optimization steps per incremental task.

4 Experiments

4.1 LIBERO Evaluation Setup

We evaluate RIFAR on the LIBERO benchmark (Liu et al., 2023) using Cosmos Policy (Kim et al., 2026), initialized from Cosmos-Predict2-2B. The policy receives third-person and wrist RGB observations, proprioceptive states, and language instructions, and predicts a 16-step action chunk.

Across the Goal, Object, and Spatial suites, we jointly train on Tasks 0–5 and then introduce Tasks 6–9 sequentially. We train all methods for 800 optimization steps per new task and evaluate all seen tasks after each stage. For RIFAR, we select the first 20 demonstrations of each past task in dataset order. We retain only the initial 16 time steps of each demonstration for replay generation. Both training phases combine current-task demonstrations with replay. Phase 1 uses the highest-quality IDM-screened trajectories for 500 steps. We then reselect trajectories from the full reliable pool based on action drift and continue training for another 300 optimization steps in Phase 2.

We compare RIFAR with sequential fine-tuning, EWC, PackNet, REGEN, and ER(n), which stores n complete demonstrations per historical task for replay. We report AUC, FWT, and NBT for each compared method, with metric definitions and the evaluation protocol in Appendix B.3.

	LIBERO-Goal			LIBERO-Object			LIBERO-Spatial		
Method	FWT $\uparrow$	NBT $\downarrow$	AUC $\uparrow$	FWT $\uparrow$	NBT $\downarrow$	AUC $\uparrow$	FWT $\uparrow$	NBT $\downarrow$	AUC $\uparrow$
Seq. FT	90.4	99.3	40.0	98.73	83.65	55.06	94.67	97.88	44.28
EWC	94.4	97.46	43.77	98.33	82.95	55.07	93.87	97.9	44.07
PackNet	76.4	88.87	31.75	98.73	81.63	56.04	81.07	93.91	37.44
ER(3)	94.8	15.81	85.74	98.73	9.51	93.44	88.27	10.66	83.13
ER(10)	93.6	4.92	90.70	98.73	2.21	97.52	92.67	3.52	90.81
ER(50)	94.4	4.62	91.8	99.13	-0.49	99.35	90.67	-0.28	90.22
REGEN	93.6	46.7	67.8	98.33	10.67	92.44	89.47	20.38	78.69
RIFAR	95.2	7.45	90.97	99.13	2.08	97.83	92.67	7.96	88.62

Table 1: Continual learning performance on LIBERO benchmarks. ER(n) denotes rehearsal with n complete demonstrations per historical task, while RIFAR stores compact initial-state anchors. All results are averaged over three random seeds. Bold values indicate the best performance for each metric among all reported methods.

4.2 Main Results and Memory Efficiency

As shown in Table 1, RIFAR achieves higher AUC and lower NBT than REGEN across all three evaluated LIBERO suites, indicating better retention of previously learned skills. On LIBERO-Goal, RIFAR improves AUC from 67.8 to 90.97 and reduces NBT from 46.7 to 7.45 compared with REGEN. The smaller AUC gains on Object and Spatial are 5.39 and 9.93 points, respectively.

RIFAR also remains competitive with experience replay using stored demonstrations, while retaining only compact initial-state anchors rather than complete trajectories. We further examine the relationship between retained historical data and performance in a separate study on LIBERO-Goal (Figure 3(b,c)). RIFAR achieves an AUC of 90.97, slightly exceeding ER(10)’s 90.70 while retaining approximately one-quarter as many historical time steps (320 versus 1,300). ER(50) achieves a higher AUC of 91.80 but retains approximately 6,500 time steps per task. These results show that RIFAR remains competitive with experience replay while retaining substantially less historical data.

4.3 Ablation Studies

Effect of the Number of Anchors. We vary the number of anchors per historical task on LIBERO-Goal to assess the trade-off between retained context and continual learning performance (Figure 3(a)). AUC increases from 80.5 at $K = 5$ to 86.8 at $K = 10$ and 90.97 at $K = 20$, then rises more slowly to 91.2 at $K = 35$ and 91.5 at $K = 50$. We therefore use $K = 20$ as our default setting for each task, which retains 40% as many anchors as $K = 50$ with AUC only 0.53 points lower.

IDM Screening and Drift-Based Selection. We compare anchor-only replay with IDM screening, drift-based selection, or both (Table 2). Either component improves AUC and reduces NBT. On LIBERO-Goal, adding drift-based selection to IDM screening raises AUC from 87.14 to 90.97 and reduces NBT from 10.61 to 7.45. RIFAR achieves the best AUC and NBT across all three suites.

Analysis of Replay Selection Criteria. Table 3 compares high-ranked, randomly sampled, and low-ranked trajectories for replay at stage 6 of each LIBERO suite, evaluating IDM reliability and drift

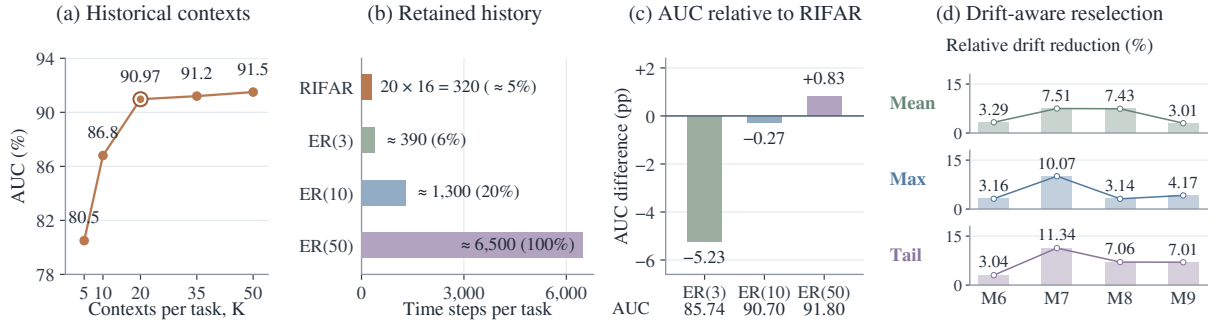


Figure 3: Historical replay and action drift on LIBERO-Goal. (a) AUC versus anchors per task (default $K = 20$). (b) Retained time steps relative to ER(50). (c) AUC differences: ER minus RIFAR (90.97). (d) Relative drift reduction on task 3 across M6–M9; positive values indicate reduced drift.

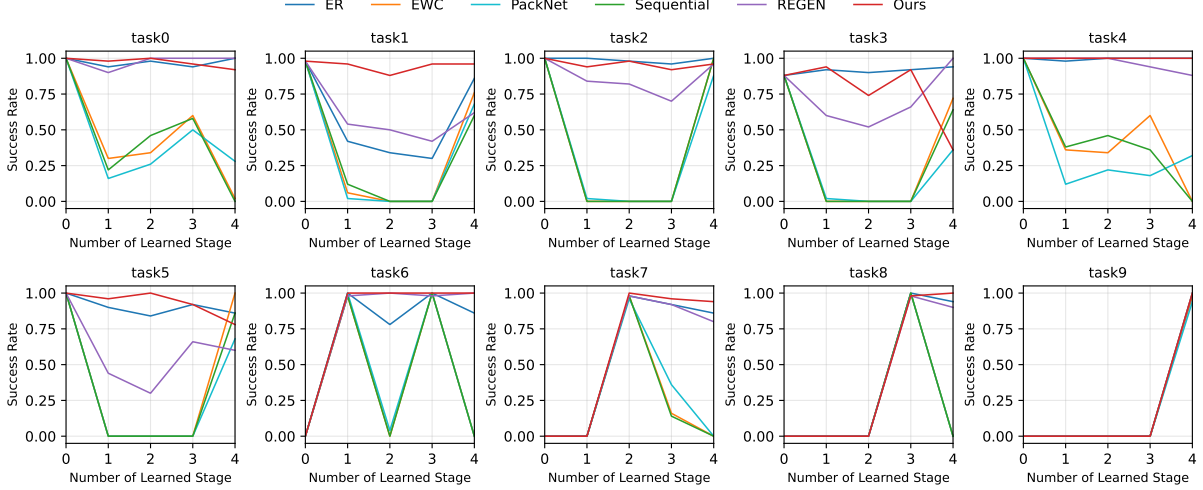


Figure 4: Per-task success rate comparison on LIBERO-Object. ER denotes ER(3). RIFAR maintains high success rates across tasks with only minor fluctuations on a small number of tasks.

Method	LIBERO-Goal			LIBERO-Object			LIBERO-Spatial		
	FWT $\uparrow$	NBT $\downarrow$	AUC $\uparrow$	FWT $\uparrow$	NBT $\downarrow$	AUC $\uparrow$	FWT $\uparrow$	NBT $\downarrow$	AUC $\uparrow$
Baseline (Anchor only)	93.0	43.5	69.5	98.33	8.79	93.27	90.01	18.37	80.36
w/ IDM	93.2	10.61	87.14	98.73	6.95	94.85	91.07	11.63	84.80
w/ Drift priority	92.8	18.18	82.49	99.53	4.14	97.06	87.07	12.81	80.59
RIFAR (full)	95.2	7.45	90.97	99.13	2.08	97.83	92.67	7.96	88.62

Table 2: Component ablation on three LIBERO suites. The baseline uses only 20 initial-state anchors for replay without IDM screening or drift-aware selection. Bold indicates the best reported value for each metric.

priority separately. High-ranked selection yields the highest success rate for both criteria in all three suites. On Goal, high-drift replay exceeds random replay by 3.1 percentage points; on Object, IDM-based selection improves success by 4.4 points.

Reliability Analysis of IDM Screening. In Figure 2(a), generated observations suggest task completion, yet executing the paired actions in the simulator fails. This case motivates checking action-visual consistency when selecting generated trajectories. IDM screening retains a subset with a 76.9% execution success rate, compared with 44.0% for unfiltered candidates, a difference of 32.9 percentage points (Figure 2(b)). The higher success rate

Stage	IDM reliability			Drift importance		
	High	Random	Low	High	Random	Low
G6	0.894	0.874	0.854	0.940	0.909	0.874
O6	0.973	0.929	0.914	0.969	0.951	0.949
S6	0.920	0.917	0.866	0.951	0.914	0.911

Table 3: LIBERO success rates on Goal (G), Object (O), and Spatial (S). High/Low denote top/bottom ranks; only replay selection varies within each strategy.

of the screened subset supports IDM agreement as a reliability signal for generated replay.

Action-Drift Analysis. We compare fixed replay and drift-aware reselection on LIBERO-Goal under the same 800-step training budget. Both variants use current demonstrations and the same screened replay set for 500 steps. The control retains this set; the other reselects the ten trajectories per historical task with the highest $F_k(\hat{\tau})$ from the full screened pool. Both then train for 300 steps with current data and their respective replay sets.

We evaluate both final policies against a shared reference using identical observations, proprioceptive states, and instructions from ten trajectories per evaluated task, with matched inference settings. Following Eq. (4), each input’s drift is the normalized ℓ_2 action difference averaged over the 16-step chunk. Mean and Max summarize the average and largest input-level drifts; Tail averages the largest 20%, ranked separately for each variant. For fixed and reselected replay statistics D_{fixed} and D_{reselect} , we define relative reduction for $D_{\text{fixed}} > 0$:

$$R_D(\%) = 100(1 - D_{\text{reselect}}/D_{\text{fixed}}). \quad (6)$$

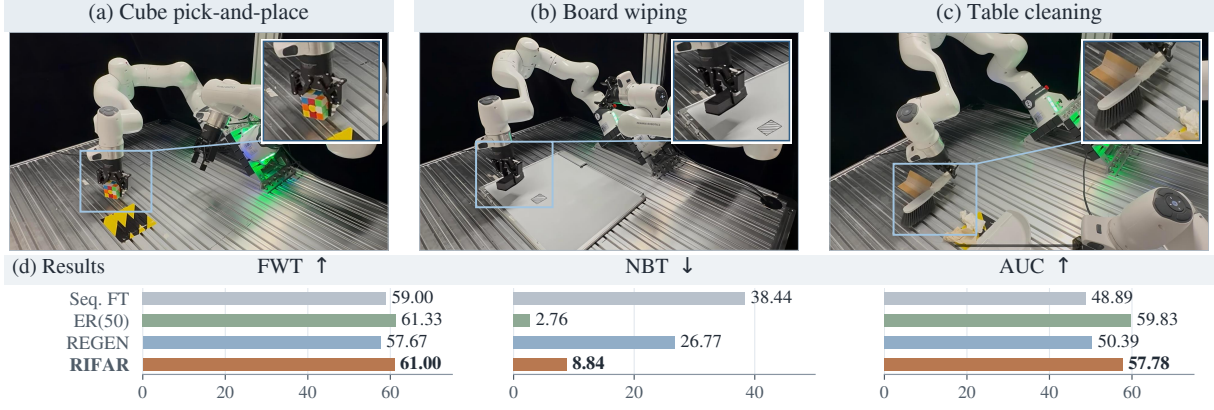


Figure 5: Real-world continual learning on a Franka robot. (a)–(c) Cube pick-and-place, board wiping, and table cleaning, with close-up insets. (d) FWT, NBT, and AUC for sequential fine-tuning (Seq. FT), ER(50), REGEN, and RIFAR (Table 4); higher FWT/AUC and lower NBT are better.

Method	FWT ↑	NBT ↓	AUC ↑
Sequential fine-tuning	59.00	38.44	48.89
ER(50)	61.33	2.76	59.83
REGEN	57.67	26.77	50.39
RIFAR (Ours)	61.00	8.84	57.78

Table 4: Real-world continual learning on three manipulation tasks using Cosmos Policy on a Franka robot. Bold indicates the best result for each metric.

Positive values indicate a reduction in action drift and negative values an increase. For task 3, Figure 3(d) shows reductions in all three statistics across M6–M9. Some other tasks show increased maximum drift (Table 9). These diagnostics measure agreement with reference actions and do not directly measure task-level forgetting.

4.4 Real-World Continual Learning

We evaluate Cosmos Policy on three real-world tasks using a Franka robot (Figure 5). Cube pick-and-place requires grasping and placement; board wiping requires controlled motion while maintaining eraser contact with the board; and table cleaning involves sweeping with a broom. Their differing motion and contact requirements test whether a shared policy can acquire new behaviors while retaining earlier skills. We compare RIFAR with sequential fine-tuning, ER(50), and REGEN. Sequential fine-tuning uses only current-task demonstrations. ER(50) replays 50 complete demonstrations per historical task, whereas REGEN and RIFAR replay generated historical experience.

After each stage, we evaluate all compared methods on all tasks seen so far and report FWT, NBT, and AUC. Table 4 shows that REGEN reduces

forgetting compared with sequential fine-tuning, while RIFAR further improves both task acquisition and retention. Relative to REGEN, RIFAR raises AUC from 50.39 to 57.78 and lowers NBT from 26.77 to 8.84. FWT also rises from 57.67 to 61.00, supporting improved new-task acquisition. Although ER(50) remains stronger overall, RIFAR approaches its AUC (57.78 versus 59.83) while retaining only compact historical contexts rather than complete original demonstrations. These results support reliable, targeted generative replay as an effective way to preserve earlier manipulation skills while retaining only compact historical contexts.

5 Conclusion

RIFAR combines reliability screening and drift-aware replay for continual WAM learning. It reconstructs past-task trajectories from compact historical contexts and uses a frozen inverse-dynamics model to assess action–visual consistency. Training first combines current demonstrations with the highest-quality screened trajectories, then prioritizes high-drift trajectories from the same reliable pool. Across three LIBERO suites and real-world experiments on a Franka robot, RIFAR achieves higher AUC and lower NBT than REGEN while remaining competitive with real-data replay. On LIBERO-Goal, it reaches 90.97 AUC while retaining only 320 historical time steps per task—4.9% of those retained by ER(50). This slightly exceeds the 90.70 AUC of ten-demonstration replay, which retains approximately 1,300 steps per task. These results support using WAMs’ generative capacity to preserve earlier manipulation skills through reliable reconstruction and adaptive replay.

Limitations

Although RIFAR improves continual learning performance, several limitations remain. First, IDM-based screening improves the quality of generated replay but cannot eliminate unsuccessful trajectories; residual errors may introduce noise into replay training. The IDM is pretrained and kept frozen during continual learning, so substantial task distribution shifts may reduce its ability to assess unfamiliar manipulation behaviors. Stronger verification mechanisms or more robust trajectory-quality models could improve reliability estimation. Second, the current WAM relies on value-based termination during autoregressive trajectory generation. A high predicted value can trigger premature stopping before the task is complete. Explicit success verification could help determine whether a generated trajectory achieves the task objective. Finally, recurrent generation can accumulate errors over long horizons. Anchor-based or hierarchical generation may improve replay quality for longer and more complex manipulation tasks.

References

- Ziyang Chen, Shaoguang Wang, Weiyu Guo, Qianyi Cai, He Zhang, Pengteng Li, Yiren Zhao, and Yandong Guo. 2026. [PHASER: Phase-aware and semantic experience replay for vision-language-action models](#). *arXiv preprint arXiv:2606.03598*.
- Manish Kumar Govind, Dominick Reilly, Smit Patel, Hieu Le, and Srijan Das. 2026. [World action models enable continual imitation learning with recurrent generative replays](#). *arXiv preprint arXiv:2606.27374*.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. [LoRA: Low-rank adaptation of large language models](#). *arXiv preprint arXiv:2106.09685*.
- Jiaheng Hu, Jay Shim, Chen Tang, Yoonchang Sung, Bo Liu, Peter Stone, and Roberto Martin-Martin. 2026. [Simple recipe works: Vision-language-action models are natural continual learners with reinforcement learning](#). *arXiv preprint arXiv:2603.11653*.
- Moo Jin Kim, Yihuai Gao, Tsung-Yi Lin, Yen-Chen Lin, Yunhao Ge, Grace Lam, Percy Liang, Shuran Song, Ming-Yu Liu, Chelsea Finn, and Jinwei Gu. 2026. [Cosmos policy: Fine-tuning video models for visuomotor control and planning](#). In *International Conference on Learning Representations*, pages 71531–71552.
- James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. 2017. [Overcoming catastrophic forgetting in neural networks](#). *Proceedings of the National Academy of Sciences*, 114(13):3521–3526.
- Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. 2023. [LIBERO: Benchmarking knowledge transfer for lifelong robot learning](#). In *Advances in Neural Information Processing Systems*, volume 36, pages 44776–44791. Curran Associates, Inc.
- Zheng Lu, Haoran Liao, Wanqi Zhong, Yunhe Ni, Lijie Wang, Xingjie Fan, Zhisheng Chen, Yantang Qu, Meijia Chen, Tianyu Xin, et al. 2026. [Layer-route: Action-conditioned mixture-of-layers routing for vision-language-action policies](#). *arXiv preprint arXiv:2609.06079*.
- Arun Mallya and Svetlana Lazebnik. 2017. [PackNet: Adding multiple tasks to a single network by iterative pruning](#). *arXiv preprint arXiv:1711.05769*.
- Ralf Römer, Yi Zhang, and Angela P. Schoellig. 2026. [CLARE: Continual learning for vision-language-action models via autonomous adapter routing and expansion](#). *arXiv preprint arXiv:2601.09512*.
- Hanul Shin, Jung Kwon Lee, Jaehong Kim, and Jiwon Kim. 2017. [Continual learning with deep generative replay](#). In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc.
- Qixin Zeng, Shuo Zhang, Hongyin Zhang, Renjie Wang, Han Zhao, Libang Zhao, Runze Li, Donglin Wang, and Chao Huang. 2026. [CRL-VLA: Continual vision-language-action learning](#). *arXiv preprint arXiv:2602.03445*.
- Libang Zhao, Qixin Zeng, Hongyin Zhang, and Donglin Wang. 2026. [Information-theoretic constraints for continual vision-language-action alignment](#). *arXiv preprint arXiv:2603.13335*.
- Chuning Zhu, Raymond Yu, Siyuan Feng, Benjamin Burchfiel, Paarth Shah, and Abhishek Gupta. 2025. [Unified world models: Coupling video and action diffusion for pretraining on large robotic datasets](#). In *Proceedings of Robotics: Science and Systems*, Los Angeles, CA, USA.
- Jiarun Zhu, Yijun Hong, Xiaoquan Sun, Zetian Xu, Qijun He, Haijier Chen, Zhiyong Wang, Mingqi Yuan, Wenjun Zeng, and Jiayu Chen. 2026. [Can vision-language-action models learn from real-world data continually without forgetting?](#) *arXiv preprint arXiv:2605.26820*.

A Implementation Specification

A.1 LIBERO Benchmark

LIBERO is a language-conditioned robot manipulation benchmark designed to evaluate lifelong robot learning under sequential task adaptation. It contains multiple task suites with different sources of task variation. In this work, we evaluate RIFAR on three LIBERO suites: LIBERO-Goal, LIBERO-Object, and LIBERO-Spatial.

LIBERO-Goal focuses on goal-level manipulation variations, where the robot needs to learn diverse manipulation objectives, such as object placement, drawer operation, and object interaction. LIBERO-Object evaluates object-level generalization by requiring similar manipulation skills to be transferred across different object categories. LIBERO-Spatial emphasizes spatial reasoning, where the robot needs to perform manipulation under different object locations and spatial configurations.

Each suite contains 10 language-conditioned manipulation tasks. Following our continual learning protocol, Tasks 0–5 are jointly used for initial policy training, while Tasks 6–9 are sequentially introduced during continual learning. The complete task instructions used in our experiments are listed in Table 5.

B Training Details

B.1 Training Protocol

Before continual learning, we first train a base policy on the initial task set of each LIBERO suite. The policy is initialized from the pretrained Cosmos-Predict2-2B-Video2World checkpoint and fine-tuned using the Cosmos-Policy framework. For each LIBERO suite, Tasks 0–5 are jointly used for base policy training, providing the initial manipulation capabilities for subsequent continual learning experiments. The training data consists of rollout trajectories. For each task, 50 successful trajectories are used during training. Demonstrations and rollout trajectories are sampled with equal probability during optimization. The policy predicts action chunks with a horizon of $H = 16$.

After obtaining the base policy checkpoint, we perform sequential continual learning on Tasks 6–9. At each continual learning stage, the model is trained using newly introduced task demonstrations together with historical replay trajectories. Each continual learning stage consists of two optimiza-

tion periods separated by a replay update step, with a total of 800 optimization steps. The first optimization period contains 500 steps. During the first 300 steps, the ratio between current-task data and replay data is set to 6:4. For the remaining 200 steps, the ratio is changed to 5:5. After this period, the historical replay set is updated using drift-aware replay selection. The model is then further optimized for 300 steps using the updated replay set with a current-task to replay-data ratio of 5:5.

B.2 Optimization Hyperparameters

The main optimization configurations used in base policy training and continual learning are summarized in Table 6. The same configuration is applied across all LIBERO suites unless otherwise specified.

B.3 Evaluation Protocol

We evaluate continual learning performance following the sequential task adaptation protocol described in the main paper. For each LIBERO suite, the model is first trained on the initial task set (Tasks 0–5) and evaluated to obtain the initial performance. Then, new tasks (Tasks 6–9) are introduced sequentially, and the model is evaluated after each task adaptation stage. During evaluation, the model is tested on all previously encountered tasks, including both the newly acquired task and historical tasks. Therefore, each evaluation stage measures both the acquisition ability on the current task and the retention of previously learned manipulation skills. All compared methods follow the same task order, evaluation environments, language instructions, and observation modalities to ensure a fair comparison.

For continual learning evaluation, we report three complementary metrics: forward transfer (FWT), normalized backward transfer (NBT), and area under the continual learning curve (AUC). FWT measures the ability of a model to acquire new tasks after continual updates. NBT evaluates the performance degradation of previously learned tasks after learning new tasks. AUC summarizes the overall continual learning performance throughout the sequential adaptation process by considering both task acquisition and knowledge retention. Higher FWT and AUC values indicate better continual learning performance, while lower NBT values indicate less degradation.

The corresponding formulations are given below. Let $r_{n,n}$ denote the performance on stage n after

Suite	Task ID	Language Instruction
LIBERO-Goal	0	open the middle drawer of the cabinet
	1	put the bowl on the stove
	2	put the wine bottle on top of the cabinet
	3	open the top drawer and put the bowl inside
	4	put the bowl on top of the cabinet
	5	push the plate to the front of the stove
	6	put the cream cheese in the bowl
	7	turn on the stove
	8	put the bowl on the plate
	9	put the wine bottle on the rack
LIBERO-Object	0	pick up the alphabet soup and place it in the basket
	1	pick up the cream cheese and place it in the basket
	2	pick up the salad dressing and place it in the basket
	3	pick up the bbq sauce and place it in the basket
	4	pick up the ketchup and place it in the basket
	5	pick up the tomato sauce and place it in the basket
	6	pick up the butter and place it in the basket
	7	pick up the milk and place it in the basket
	8	pick up the chocolate pudding and place it in the basket
	9	pick up the orange juice and place it in the basket
LIBERO-Spatial	0	pick up the black bowl between the plate and the ramekin and place it on the plate
	1	pick up the black bowl next to the ramekin and place it on the plate
	2	pick up the black bowl from table center and place it on the plate
	3	pick up the black bowl on the cookie box and place it on the plate
	4	pick up the black bowl in the top drawer of the wooden cabinet and place it on the plate
	5	pick up the black bowl on the ramekin and place it on the plate
	6	pick up the black bowl next to the cookie box and place it on the plate
	7	pick up the black bowl on the stove and place it on the plate
	8	pick up the black bowl next to the plate and place it on the plate
	9	pick up the black bowl on the wooden cabinet and place it on the plate

Table 5: Task instructions used in LIBERO continual learning experiments. Tasks 0–5 are used for initial joint training, while Tasks 6–9 are sequentially introduced during continual learning.

learning stage n , and $r_{p,n}$ denote the performance on stage p after learning stage n . FWT is computed as

$$\text{FWT} = N^{-1} \sum_{n=1}^N r_{n,n}. \quad (7)$$

NBT measures the normalized performance degradation of previous stages:

$$\text{NBT}_n = \frac{1}{N-n} \sum_{p=n+1}^N \frac{r_{n,n} - r_{p,n}}{r_{n,n}}, \quad (8)$$

and the overall NBT is averaged over all previous stages:

$$\text{NBT} = \frac{1}{N-1} \sum_{n=1}^{N-1} \text{NBT}_n. \quad (9)$$

Finally, AUC summarizes the overall continual learning performance:

$$\text{AUC} = \frac{1}{N} \sum_{n=1}^N \frac{r_{n,n} + \sum_{p=n+1}^N r_{p,n}}{N-n+1}. \quad (10)$$

Here, N denotes the total number of sequential learning stages, and $r_{p,n}$ represents the performance of stage p after the model has been trained up to stage n .

In addition to aggregate metrics, we report per-task success rates to provide a more detailed analysis of task-wise performance. For each task, success is determined according to the standard LIBERO evaluation criterion, where an episode is considered successful only when the specified manipulation objective is completed. Detailed task-level results are provided in the supplementary quantitative analysis. All experiments are repeated with three random seeds, and the reported results are averaged across seeds. The same evaluation protocol is applied to LIBERO-Goal, LIBERO-Object, and LIBERO-Spatial.

Category	Hyperparameter	Value
Model	Backbone	Cosmos-Predict2-2B-Video2World
	Framework	Cosmos-Policy
	Action horizon	16
Base training	GPUs	8
	Batch size per GPU	15
	Gradient accumulation	2
	Effective batch size	240
	Learning rate	1×10^{-4}
	Training iterations	30K
Continual learning	GPUs	8
	Batch size per GPU	15
	Gradient accumulation	3
	Total optimization steps/task	800
	Warm-up steps	50
	Peak learning rate	3.5×10^{-5}
	Final learning rate	2.5×10^{-5}

Table 6: Optimization hyperparameters used in training.

C Additional Results

C.1 Task-wise Continual Learning Performance

To provide a detailed view of continual learning behavior, we report the task-wise success rate matrices on the three LIBERO suites. Each matrix records the success rates evaluated after each continual learning stage. Rows correspond to evaluation stages, while columns correspond to previously encountered tasks. Therefore, the matrices reflect both the acquisition of newly introduced tasks and the retention of historical manipulation skills.

For compact visualization, we represent the performance evolution as 5×5 matrices. As shown in Figure 6, the first row corresponds to the initial policy after joint training on Tasks 0–5, and the following rows correspond to sequential adaptation after learning new tasks (Tasks 6–9). All success rates are averaged over three random seeds. The same evaluation protocol is applied to LIBERO-Goal, LIBERO-Object, and LIBERO-Spatial.

C.2 IDM-based Trajectory Reliability Scoring

We use an IDM-based scoring model to evaluate replay trajectory reliability. The IDM checkpoint used in our experiments is `outputs/best.pt`. The model predicts 16-step action chunks with an action dimension of 7. For each chunk, the normalized action prediction error is

$$E_{\text{IDM}} = d^{-1/2} \text{mean}_t \|\hat{a}_t - a_t\|_2, \quad (11)$$

where $\hat{a}_t$ and a_t are the normalized predicted and executed action vectors and $d = 7$. This is the root-mean-square error across action dimensions, averaged over time. The checkpoint-calibrated reliability score is

$$Q_{\text{IDM}} = \sigma((b - sE_{\text{IDM}})/T), \quad (12)$$

with $s = 8.6402692889$, $b = 6.5952286129$, and $T = 1.0$. These values specify the same sigmoid mapping as Eq. (3), with $\alpha = b/T$ and $\beta = s/T$. Lower prediction errors therefore produce higher IDM scores.

For trajectory-level scoring, we first smooth the scores using a three-chunk trailing moving average. For N chunk scores, let $Q_{(j)}^{\text{sm}}$ denote the j -th smallest smoothed value and set $k = \max(1, \lceil 0.2N \rceil)$. The trajectory soft score is the mean of the lowest 20%:

$$Q_{\text{soft}} = k^{-1} \sum_{j=1}^k Q_{(j)}^{\text{sm}}. \quad (13)$$

Here, Q_{soft} corresponds to `soft_score` in the implementation. This design emphasizes locally unreliable segments instead of averaging out short-term failures.

A trajectory is classified as reliable when $Q_{\text{soft}} \geq 0.65$ and the longest consecutive sequence with $Q_{\text{IDM}} < 0.40$ is shorter than 3 chunks. For drift evaluation, chunks with $Q_{\text{IDM}} < 0.60$ are regarded as low-score drift chunks, and the allowed number is $\max(1, \lfloor 0.2N_{\text{judged}} \rfloor)$. Consecutive drift length is only recorded for analysis and is not used as an additional rejection condition.

Continual Learning Performance on LIBERO Benchmarks																	
LIBERO Goal						LIBERO Object						LIBERO Spatial					
	S0	S1	S2	S3	S4		S0	S1	S2	S3	S4		S0	S1	S2	S3	S4
Model 1	0.96	—	—	—	—	Model 1	0.977	—	—	—	—	Model 1	0.973	—	—	—	—
Model 2	0.95	0.88	—	—	—	Model 2	0.963	1	—	—	—	Model 2	0.94	1	—	—	—
Model 3	0.91	0.8	1	—	—	Model 3	0.933	1	1	—	—	Model 3	0.9	0.96	0.8	—	—
Model 4	0.747	0.78	1	1	—	Model 4	0.947	1	0.96	0.98	—	Model 4	0.867	0.92	0.74	0.9	—
Model 5	0.7	0.64	1	1	0.92	Model 5	0.853	1	0.94	1	1	Model 5	0.833	0.9	0.72	0.84	0.9

Figure 6: Continual learning performance on LIBERO-Goal, LIBERO-Object, and LIBERO-Spatial. Rows represent model checkpoints after successive learning stages, and columns denote task groups S0–S4. Each cell reports the corresponding task success rate, while not-yet-introduced tasks are masked. The lower-triangular matrices visualize both new-task acquisition and retention of previously learned tasks. Results are averaged over three random seeds.

Data	Setting	Task0	Task1	Task2	Task3	Task4	Task5	Task6	Task7	Task8
Goal	Model6	36/37	40/40	40/40	34/34	40/40	30/33	—	—	—
	Model7	36/36	40/40	33/34	24/24	40/40	28/31	38/38	—	—
	Model8	25/31	40/40	35/35	13/13	37/37	6/15	35/35	39/39	—
	Model9	14/24	40/40	33/35	33/33	33/34	2/20	30/33	36/37	40/40
Object	Model6	30/33	34/36	34/35	35/36	34/35	30/33	—	—	—
	Model7	21/26	27/29	28/30	30/32	24/29	21/27	23/26	—	—
	Model8	19/24	22/26	20/24	27/30	25/30	3/20	10/22	30/32	—
	Model9	7/22	4/20	10/23	21/24	4/20	2/21	3/20	23/28	25/29
Spatial	Model6	35/36	39/39	38/38	40/40	22/28	32/33	—	—	—
	Model7	35/35	35/35	30/31	37/37	23/28	37/37	34/35	—	—
	Model8	22/28	34/34	33/34	28/30	19/26	16/24	35/35	32/33	—
	Model9	24/30	37/37	19/25	21/27	17/28	5/22	25/26	33/33	30/31

Table 7: Reliable trajectory counts after IDM-based filtering across different continual learning settings. Each entry denotes the number of trajectories satisfying the IDM reliability criterion over the total trajectories.

If the strict reliable condition is not satisfied, trajectories with $Q_{\text{soft}} \geq 0.40$, fewer than four consecutive low-score chunks, and a passed drift gate are categorized as suspect; otherwise they are classified as unreliable. The number of trajectories satisfying the reliable criterion after IDM-based filtering for each continual learning setting is reported in Table 7. When fewer than 10 reliable trajectories are selected for a task, the selected trajectories are sequentially duplicated until the replay set reaches the required size of 10 trajectories.

C.3 Drift Analysis across Continual Learning Stages

To further analyze the effect of drift-aware replay, we measure the behavioral drift of different continual learning models with and without drift-aware selection. D− denotes the replay strategy with-

out drift-aware selection, while D+ denotes the proposed drift-aware replay strategy. For Task0–Task5, the drift is calculated between each continual learning model and the corresponding base model. For Task6 and later stages, the drift is measured between consecutive continual learning models, reflecting the additional behavioral changes introduced by subsequent task learning.

As summarized in Table 9, drift-aware replay consistently reduces the behavioral drift compared with standard replay across different continual learning stages. The reductions in Mean, Max, and Tail drift indicate that prioritizing drift-sensitive trajectories helps preserve previous behaviors during continual updates, thereby mitigating catastrophic forgetting.

Dataset	Setting	Task0	Task1	Task2	Task3	Task4	Task5	Task6	Task7	Task8
Goal	Model6	4	1	3	2	3	1	–	–	–
	Model7	2	1	0	5	3	3	6	–	–
	Model8	6	1	5	8	2	10	2	2	–
	Model9	6	2	3	3	2	10	3	3	2
Object	Model6	3	3	1	2	4	2	–	–	–
	Model7	6	3	3	3	3	3	3	–	–
	Model8	5	3	5	3	4	10	8	5	–
	Model9	10	10	10	6	10	10	10	6	5
Spatial	Model6	2	5	2	2	4	4	–	–	–
	Model7	4	3	4	3	3	5	4	–	–
	Model8	3	2	3	5	6	10	6	4	–
	Model9	2	5	5	3	7	10	5	2	2

Table 8: Overlap of selected trajectories between replay1 and replay2 after the final replay selection process. Each entry denotes the number of shared trajectories between the two replay sets.

C.4 Replay Selection Overlap Analysis

To analyze the consistency between different replay selection stages, we report the number of overlapping trajectories between replay1 and replay2 after the final replay selection process. Each entry in Table 8 denotes the number of shared trajectories between the two replay sets.

The overlap statistics characterize the consistency of the replay selection pipeline. The shared trajectories indicate samples consistently considered valuable by both selection stages, while the remaining trajectories reflect the additional filtering effect introduced by subsequent selection criteria.

Stage	Model Pair	D−			D(+)		
		Mean	Max	Tail	Mean	Max	Tail
Task0	M6 / Base	0.4702	1.0278	0.9986	0.4622	1.2278	0.9851
	M7 / Base	0.8844	1.5086	1.3399	0.8732	1.5630	1.3642
	M8 / Base	1.0830	1.8404	1.6377	0.9980	1.7454	1.5116
	M9 / Base	1.2368	2.1180	1.9027	1.1925	2.1393	1.8564
Task1	M6 / Base	0.5246	1.3363	0.9653	0.4849	1.0559	0.8336
	M7 / Base	0.9532	2.3010	1.8192	0.9005	2.1672	1.7005
	M8 / Base	1.1439	2.2692	1.7995	1.0640	2.1126	1.6757
	M9 / Base	1.1319	1.9630	1.6588	1.0826	1.9638	1.5919
Task2	M6 / Base	0.7299	2.0123	1.5149	0.6481	1.4927	1.1938
	M7 / Base	1.3450	3.1758	2.3791	1.0817	2.0187	1.6307
	M8 / Base	1.4428	3.1799	2.5013	1.4083	3.0468	2.4145
	M9 / Base	1.4861	3.0998	2.5764	1.4444	2.9463	2.4679
Task3	M6 / Base	0.6780	1.8929	1.4150	0.6557	1.8331	1.3720
	M7 / Base	0.9599	2.5190	1.8091	0.8878	2.2653	1.6039
	M8 / Base	1.0848	2.7887	2.0037	1.0042	2.7010	1.8623
	M9 / Base	1.1851	3.0954	2.2025	1.1494	2.9663	2.0481
Task4	M6 / Base	0.8072	3.0531	1.9174	0.5707	1.5384	1.1316
	M7 / Base	0.8777	2.2010	1.5212	0.7995	1.1684	1.2225
	M8 / Base	1.2093	2.8327	2.1253	1.1053	2.3763	1.8339
	M9 / Base	1.2180	2.6270	2.0648	1.1831	2.5046	1.9428
Task5	M6 / Base	0.5987	1.7364	1.3820	0.5303	1.3084	1.0716
	M7 / Base	1.0681	2.7923	2.1518	0.9932	2.5452	1.9652
	M8 / Base	1.0112	2.4939	1.9570	0.9932	2.4285	1.8829
	M9 / Base	0.9194	2.1695	1.8226	0.8871	2.0902	1.7546
Task6	M7 / M6	0.5947	1.0402	0.9046	0.5608	0.9951	0.8512
	M8 / M6	0.8979	1.5276	1.3767	0.8511	1.6065	1.3636
	M9 / M6	1.0569	1.7464	1.5407	1.0127	1.6696	1.5016
Task7	M8 / M7	0.9610	2.1996	1.6654	0.8554	2.1114	1.6159
	M9 / M7	1.2095	2.3265	2.0775	1.1226	2.0307	1.8673
Task8	M9 / M8	0.7177	1.7683	1.3203	0.6572	1.3997	1.1490

Table 9: Detailed action drift comparison across continual learning stages. The model pair denotes the two models used to calculate behavioral drift. D− denotes replay without drift-aware selection, while D(+) denotes replay with drift-aware selection. Mean, Max, and Tail represent the average drift, maximum drift, and the average drift of the top 20% highest-drift samples, respectively.